Sitewire Interview Notes

Simple guide to explain every file in your app.

Read this on your phone before the interview.

What does the app do?

It shows a table of users from an API. For each user it also fetches their last login time and IP address.

The API is unreliable - about 1 in 4 requests fail, and some are slow. The app handles loading, success, and error states clearly.

Core: ID, name, email, last login time, last login IP, total user count.

Bonus: friendly dates, country from IP, highlight inactive users.

How to demo (say this on the call)

1. Open Dashboard - table shows first, login info loads row by row
2. Point at status banner - user count + login progress (45/100)
3. Show a Failed to load row if you see one - one user failing does not break the page
4. Open Load log - step-by-step timeline of what happened
5. Switch to Core + Bonus tab - show friendly dates, country, inactive highlighting
6. If they ask about code - open hooks/useEnrichedUsers.ts first

The story of one page load

1. App opens. useEnrichedUsers runs automatically.
2. Ask API for all users. If that fails - show error + Retry button.
3. Users arrive - show table immediately. Login cells say Loading...
4. Fetch logins for 15 users at a time (not all 100 at once).
5. For each user: get logins, pick newest, look up country, update that row.
6. Repeat until done.

Every file explained

index.html

What it does: The empty webpage. Has a div called root where React draws the app. Loads main.tsx as a script.

Say in interview: Think of it as an empty picture frame.

src/main.tsx

What it does: Entry point - first code that runs. Finds the root div and puts the App component inside it.

Say in interview: main.tsx bootstraps the app.

src/App.tsx

What it does: Boss component. Controls tabs (Core/Bonus, Explain/Dashboard/Load log). Calls useEnrichedUsers for data and useEventLog for the timeline.

Say in interview: App.tsx is layout and tabs. Data logic is in the hook.

src/types.ts

What it does: Describes data shapes. User = basic info. EnrichedUser = User + login fields + status. LoginStatus = loading, success, or error per row.

Say in interview: EnrichedUser is what each table row uses.

src/types/events.ts

What it does: Types for the Load log events (batch started, batch done, etc). Demo helper only.

Say in interview: Only used for the Load log walkthrough tab.

src/api/client.ts

What it does: fetchWithRetry - sends requests to the API. On 500 error: wait 500ms, retry up to 3 times.

Say in interview: Handles the flaky API the challenge describes.

src/api/users.ts

What it does: Two functions: getUsers() and getUserLogins(id). Thin wrapper around API endpoints.

Say in interview: Just the two API calls the challenge needs.

src/api/geo.ts

What it does: Bonus. Turns IP into country using ipwho.is. Caches results so same IP is not looked up twice.

Say in interview: Geo failure shows dash, does not break the row.

src/hooks/useEnrichedUsers.ts

What it does: THE BRAIN. Loads users, then logins in batches of 15. Updates each row. Uses fetchIdRef so old requests are ignored on Retry.

Say in interview: Start here when they say walk me through the code.

src/hooks/useEventLog.ts

What it does: Records timeline events for Load log tab. beginRecording, recordEvent, copyNotes.

Say in interview: Helps me demo what the app is doing step by step.

src/utils/login.ts

What it does: getMostRecentLogin - finds newest login (API does not sort). formatLoginTimeHumanized - 3 months ago. isInactiveOverOneMonth - for bonus highlighting.

Say in interview: Small pure functions, easy to test.

src/components/StatusBanner.tsx

What it does: Box above table. Shows Loading users, or error + Retry, or user count + login progress.

Say in interview: Global state for the /users call.

src/components/UserTable.tsx

What it does: Draws the table headers and one UserRow per user. Core vs bonus variant toggles columns.

Say in interview: Just the table structure.

src/components/UserRow.tsx

What it does: One table row. Shows Loading, Failed to load, or actual data based on loginStatus.

Say in interview: Per-row state - this is partial success.

src/components/LoadLogPanel.tsx

What it does: Load log tab UI - progress bar, event list, copy button.

Say in interview: Great for explaining async flow on the call.

src/components/CoreTaskExplain.tsx

What it does: Explain tab for core task - requirements summary and live stats.

Say in interview: Built-in cheat sheet during demo.

src/components/BonusTaskExplain.tsx

What it does: Explain tab for bonus features.

Say in interview: Shows what the 3 bonuses do.

src/components/ExplainSectionCard.tsx

What it does: Reusable titled box for explain panels. Just UI wrapper.

Say in interview: No logic - keeps panels consistent.

src/App.css and src/index.css

What it does: Styling - colors, tabs, table, banners. Challenge said do not focus on polish.

Say in interview: Just makes it readable.

If they ask...

Why batches of 15?
100 requests at once would overwhelm a flaky API. Batches let rows fill in gradually so the user sees progress.

/users fails vs one login fails?
/users fail = whole page error. One login fail = just that row shows Failed to load.

How find most recent login?
Loop and compare dates. API sends logins in random order.

What is a hook?
A React function that holds state and logic. useEnrichedUsers holds users and the load function.

What is fetchIdRef?
A counter bumped on each reload. Old network responses check it and ignore themselves if stale.

Why not React Query?
Small app - wanted retry and batch logic visible. In production I would use a library.

Good luck tomorrow!